Concurrency: Mutual exclusion and synchronization.



Some key words

- Multiprogramming
  - The management of multiple processes within a uniprocessor system.
- Multiprocessing
  - The management of multiple processes within a multiprocessor.
- Distributed processing:
  - The management of multiple processes executing on multiple, distributed computer systems.

Concurrency

- Ability to run multiple processes or tasks simultaneously or in an overlapping manner.
- Concurrency and Parallelism are related concepts **but not the same.**
- **In parallel execution, processes are truly executed simultaneously.**
- **Parallelism is type of concurrency.**

Difficulties of Concurrency

- Sharing of global resources.
- Difficult for the OS to manage the allocation of resources optimally.
- Difficult to locate programming errors as results are not deterministic and reproducible.

Race Condition

- Occurs when multiple processes or threads access shared data concurrently, with at least one modifying it.
- The final result depends on the timing or order of their execution.
- Without proper synchronization, concurrent accesses may produce unexpected or incorrect result

Resource competition

- Concurrent processes come into conflict when they are competing for use of the same resource
  - For example: I/O devices, memory, processor time, clock

In the case of competing processes three control problems must be faced:

- **The need for mutual exclusion.**
- **Deadlocks.**
- **Starvation.**

What is mutual exclusion?

- The requirement that when one process is in a **critical section** that accesses shared resources, no other process may be in a critical section that accesses any of those shared resources.
- **Critical section**: A section of code within a process that requires access to shared resources, and that must not be executed while another process is in a corresponding section of code.

How to ensure mutual exclusion?

- Concurrency control mechanisms.
  - Software based mechanisms.
  - Hardware based mechanisms.
  - OS and programming language support.
- We assume there exist some sort of elementary mutual exclusion at memory access level.
  - Simultaneous accesses (reading and/or writing) to the same location in main memory are serializable by some sort of memory arbiter.
    - Specifically, if two processes try to read or write the same memory location simultaneously, there is a memory arbiter (hardware logic in the memory system) that ensures their accesses are serialized (happen one after another, never physically at the same exact cycle).
    - Important: This does not guarantee fairness or a specific order — it only guarantees that memory contents will not get corrupted by truly simultaneous writes.

How to ensure mutual exclusion?

- Software based approach
- Hardware support
  - Interrupt disabling
  - Atomic Instructions
    - **Compare&Swap** instruction.
    - Exchange Instruction.
- OS and programming language support
  - Mutex
  - Semaphore
  - Monitor

Software based solutions

- At first glance, it seems like enforcing mutual exclusion is easy.
- We will see why this is not the case.
- We will look at **several incorrect solutions first.**
- As we saw earlier, every mutual exclusion mechanism needs some type of hardware support.
- Suppose we have that access to memory location is serialized.

Software based solutions

- Suppose, you have two processes $P0, P1$ trying to access a shared resource.
- Let's create a global integer variable called $\boldsymbol{turn}$.
  - $int\ turn = 0$
  - we initialize it 0 in the beginning.
- If the variable $\boldsymbol{turn}$ is equal to the id of the process, then it can proceed to access the resource otherwise it waits until the value of turn is equal to its id.
  - This is called $\boldsymbol{busy\ waiting}$, or $\boldsymbol{spin\ waiting}$.
- After a process gains access to the shared resource, and after it has finished using the resource, it updates the turn variable to that of other process.

```
   /* PROCESS 0 /*                    /* PROCESS 1 *
.                                  .
.                                  .
while (turn != 0)                  while (turn != 1)
   /* do nothing */ ;                 /* do nothing */;
/* critical section*/;              /* critical section*/;
turn = 1;                          turn = 0;
.                                  .
```

(a) First attempt

Implications

- Does it guarantee mutual exclusion?
  - Yes
- Drawbacks?
  - Process 1 cannot start using the resource at the beginning.
  - Processes must strictly alternate in their use of shared resource.
    - Slower process dictates the pace of execution.
  - If one process fails, then other process is permanently blocked.

Second attempt

- The problem of first attempt was that processes did not know the intention of the other process to use the shared resource.
- Also, we did not know the state of the process—whether it is failed or not.
- So, we keep track two Boolean flags—one for each process.
  - $boolean\ flag[2] = 0, 0$
  - $P0\ has\ flag[0]$
  - $P1\ has\ flag[1]$
- Each process may examine the other's flag but not alter it.
- When ever process needs to access shared resource, it changes its own flag to **true.**

Second attempt

```
enum      boolean (false = 0; true = 1);
boolean   flag[2] = 0, 0
```

```
   /* PROCESS 0 *                /* PROCESS 1 *
.                                .
.                                .
while (flag[1])                  while (flag[0])
   /* do nothing */;                 /* do nothing */;
flag[0] = true;                  flag[1] = true;
/*critical section*/;            /* critical section*/;
flag[0] = false;                 flag[1] = false;
.                                .
```

(b) Second attempt

Implications?

- If a process fails outside of the critical section, then the other process can continue to use the shared resource.
- Does this attempt provide mutual exclusion?
  - No.
  - At the beginning, both flags are set to **false.**
  - **What if both process continue to use the shared resource?**
- **This program is incorrect.**

Third attempt

- What if we initialize both flags to **true?**

```
   /* PROCESS 0 *                /* PROCESS 1 *
.                              .
.                              .
flag[0] = true;                flag[1] = true;
while (flag[1])                while (flag[0])
   /* do nothing */;              /* do nothing */;
/* critical section*/;          /* critical section*/;
flag[0] = false;               flag[1] = false;
.                              .
```

(c) Third attempt

Implications

- Does it guarantee mutual exclusion?
  - Yes
- But there is a bigger issue.
  - Neither of these processes can enter to the critical section since, they both have set their flags to **true.**
  - **So, it is deadlocked.**
- **Not a good solution.**

Fourth attempt

- If a process cannot access the shared resource because of the other process’s desire to use the resource. Then the process 1 is prepared to reset their flag to defer to the other process.

```
   /* PROCESS 0 *                  /* PROCESS 1 *
.                                  .
.                                  .
flag[0] = true;                    flag[1] = true;
while (flag[1]) {                  while (flag[0]) {
   flag[0] = false;                   flag[1] = false;
   /*delay */;                        /*delay */;
   flag[0] = true;                    flag[1] = true;
}                                  }
/*critical section*/;              /* critical section*/;
flag[0] = false;                   flag[1] = false;
.
```

(d) Fourth attempt

Implications

- Does it guarantee mutual exclusion?
  - Yes.
- But there could be **livelock**.

Correct solution

- The problem of fourth attempt was that there was no order of activities of the two processes.
- Hence, each process allows other process to move forward by resetting their desire to use the shared resource.
- Thus, we will use the $turn$ variable to indicate which process has right to move to the critical section.

Algorithm 1 Decker's algorithm

```
1: boolean flag[2]
2: int turn
3: function P0( )
4:     while true do
5:         flag[0] ← true
6:         while flag[1] do
7:             if turn == 1 then
8:                 flag[0] = false
9:                 while turn == 1 do                ▷ Do nothing
10:                flag[0] ← true
11:        \* Acccess the shared resource */
12:        turn ← 1
13:        flag[0] ← false
14: function P1( )
15:    while true do
16:        flag[1] ← true
17:        while flag[0] do
18:            if turn == 0 then
19:                flag[1] = false
20:                while turn == 0 do                ▷ Do nothing
21:                flag[1] ← true
22:        \* Acccess the shared resource */
23:        turn ← 0
24:        flag[1] ← false
25: function MAIN( )
26:    flag[0] ← false
27:    flag[1] ← false
28:    turn ← 1
29:    parbegin(P0, P1)
```

Peterson’s algorithm

- More elegant solution.

2 Peterson's algorithm

Algorithm 2 Peterson's algorithm

```
1: boolean flag[2]
2: int turn
3: function P0( )
4:     while true do
5:         flag[0] ← true
6:         turn ← 1
7:         while flag[1] ∧ (turn == 1) do                ▷ Do nothing
8:             \* Do nothing */
9:         \* Access the shared resource */
10:        flag[0] ← false
11:        \* Remainder */
12: function P1( )
13:    while true do
14:        flag[1] ← true
15:        turn ← 0
16:        while flag[0] ∧ (turn == 0) do                ▷ Do nothing
17:            \* Do nothing */
18:        \* Access the shared resource */
19:        flag[1] ← false
20:        \* Remainder */
21: function MAIN( )
22:    flag[0] ← false
23:    flag[1] ← false
24:    parbegin(P0, P1)
```

Mutual Exclusion: Hardware Support

- Interrupt Disabling
  - In a uniprocessor system, concurrent processes cannot have overlapped execution; they can only be interleaved
  - A process will continue to run until it invokes an OS service or until it is interrupted
  - Therefore, to guarantee mutual exclusion, it is sufficient to prevent a process from being interrupted
  - This capability can be provided in the form of primitives defined by the OS kernel for disabling and enabling interrupts
- Disadvantages
  - The efficiency of execution could be noticeably degraded because the processor is limited in its ability to interleave processes
  - This approach will not work in a multiprocessor architecture

```
while (true) {
    /* disable interrupts */;
    /* critical section */;
    /* enable interrupts */;
    /* remainder */;
}
```

Mutual Exclusion: Hardware Support

- **Compare&Swap** Instruction
  - Also called a “compare and exchange instruction”
  - A compare is made between a memory value and a test value
  - If the values are the same a swap occurs
  - Carried out atomically (not subject to interruption)

```
int compare_and_swap (int *word, int testval, int newval)
{
    int oldval;
    oldval = *word
    if (oldval == testval) *word = newval;
    return oldval;
}
```

- Two version of this exist
  - It returns the old value.
  - Returns true if swap occurred, otherwise returns false.
- This function returns the old value.
  - **Remember this is an atomic instruction.**

Algorithm 3 Compare and Swap instruction based mutual exclusion

```
1: const int n ← #of processes
2: int bolt
3: function P(int i)
4:     while true do
5:         while compare_and_swap(bolt, 0, 1) == 1 do
6:             \* Do nothing */
7:         \* Access the shared resource */
8:         bolt ← 0
9:         \* Remainder */
10: function MAIN( )
11:     bolt ← 0
12:     parbegin(P(0), P(1), ..., P(n))
```

- Whichever process finds the $bolt = 0$ goes into the critical section and sets the $bolt = 1$.
- **All the other processes goes into busy waiting mode (or spin waiting).**
  - The term **busy waiting, or spin waiting**, refers to a technique in which a process can do nothing until it gets permission to enter its critical section, but continues to execute an instruction or set of instructions that tests the appropriate variable to gain entrance.

Implications

Advantages

- Easy to use.
- Easy to see why this works.
- Can be extended to multiple critical sections—define a variable for each critical section.
- Can be used in Uniprocessor or Multiprocessor system for any number of processes.

Disadvantages

- Busy waiting is used—waste CPU time.
- Starvation is possible.
  - When more than one process is waiting, the selection process is arbitrary.
- Deadlock is also possible.

OS and language support

- Mutex
- Semaphore
- Monitors
- Condition variable

Semaphores

- A semaphore may be initialized into a nonnegative value.
- The semWait operation decrements the semaphore value.
- The semSignal operation increments the semaphore value.

A variable that has an integer value upon which only three operations are defined:

- There is no way to inspect or manipulate semaphores other than these three operations

```
struct semaphore {
     int count;
     queueType queue;
};
void semWait(semaphore s)
{
     s.count--;
     if (s.count < 0) {
          /* place this process in s.queue */;
          /* block this process */;
     }
}
void semSignal(semaphore s)
{
     s.count++;
     if (s.count <= 0) {
          /* remove a process P from s.queue */;
          /* place process P on ready list */;
     }
}
```

Figure 5.6 A Definition of Semaphore Primitives

Consequences

There is no way to know before a process decrements a semaphore whether it will block or not

There is no way to know which process will continue immediately on a uniprocessor system when two processes are running concurrently

You don’t know whether another process is waiting so the number of unblocked processes may be zero or one

```
struct binary semaphore {
     enum {zero, one} value;
     queueType queue;
};
void semWaitB(binary semaphore s)
{
     if (s.value == one)
          s.value = zero;
     else {
            /* place this process in s.queue */;
            /* block this process */;
     }
}
void semSignalB(semaphore s)
{
     if (s.queue is empty())
          s.value = one;
     else {
            /* remove a process P from s.queue */;
            /* place process P on ready list */;
     }
}
```

Figure 5.7 A Definition of Binary Semaphore Primitives

Strong/weak semaphores

- A queue is used to hold processes waiting on the semaphore

Strong Semaphores

- The process that has been blocked the longest is released from the queue first (FIFO)

Weak Semaphores

- The order in which processes are removed from the queue is not specified

Implementation of Semaphores

- Imperative that the **semWait** and **semSignal** operations be implemented as atomic primitives.
- Can be implemented in hardware or firmware
- Software schemes such as Dekker's or Peterson's algorithms can be used
- Another alternative is to use one of the hardware-supported schemes for mutual exclusion

```
semWait(s)
{
    while (compare_and_swap(s.flag, 0 , 1) == 1)
        /* do nothing */;
    s.count--;
    if (s.count < 0) {
        /* place this process in s.queue*/;
        /* block this process (must also set s.flag to 0)
*/;
    }
    s.flag = 0;
}

semSignal(s)
{
    while (compare_and_swap(s.flag, 0 , 1) == 1)
          /* do nothing */;
    s.count++;
    if (s.count <= 0) {
        /* remove a process P from s.queue */;
        /* place process P on ready list */;
    }
    s.flag = 0;
}
```

(a) Compare and Swap Instruction

```
semWait(s)
{
    inhibit interrupts;
    s.count--;
    if (s.count < 0) {
        /* place this process in s.queue */;
        /* block this process and allow interrupts */;
    }
    else
       allow interrupts;
}

semSignal(s)
{
    inhibit interrupts;
    s.count++;
    if (s.count <= 0) {
        /* remove a process P from s.queue */;
        /* place process P on ready list */;
    }
    allow interrupts;
}
```

(b) Interrupts

Figure 5.17 Two Possible Implementations of Semaphores

Mutex (Mutual exclusion lock)

- Binary semaphore and mutex are similar.
- But they are not the same.
- You have two operations.
  - Mutex.lock()
  - Mutex.unlock()
- The process that locks the mutex should be the process that unlocks the mutex as well.
- In a binary semaphore, this is not the case.

Monitors

- **Programming language construct** that provides equivalent functionality to that of semaphores and is easier to control.
- Implemented in a number of programming languages
    - Concurrent Pascal, Pascal-Plus, Modula-2, Modula-3, Java
- Has also been implemented as a program library.
- Software module consisting of one or more procedures, an initialization sequence, and local data.
- If you have a data structure that needs mutual exclusion, you can wrap the data structure with a monitor.
    - In java you can use:
        - $synchronized(someObject)\ \{do\ something\}$

Monitor characteristics

1. The local data variables are accessible only by the monitor's procedures and not by any external procedure.
2. A process enters the monitor by invoking one of its procedures.
3. **Only one process may be executing in the monitor at a time;** any other processes that have invoked the monitor are blocked, waiting for the monitor to become available

Monitor characteristics

Local data variables are accessible only by the monitor's procedures and not by any external procedure

Process enters monitor by invoking one of its procedures

Only one process may be executing in the monitor at a time

Synchronization

- A monitor supports synchronization by the use of **condition variables** that are contained within the monitor and accessible only within the monitor
    - Condition variables are a special data type in monitors which are operated on by two functions:
        - `cwait(c)`: suspend execution of the calling process on condition `c`. Monitor is then available for use by another process.
        - `csignal(c)`: resume execution of some process blocked after a `cwait` on the same condition.

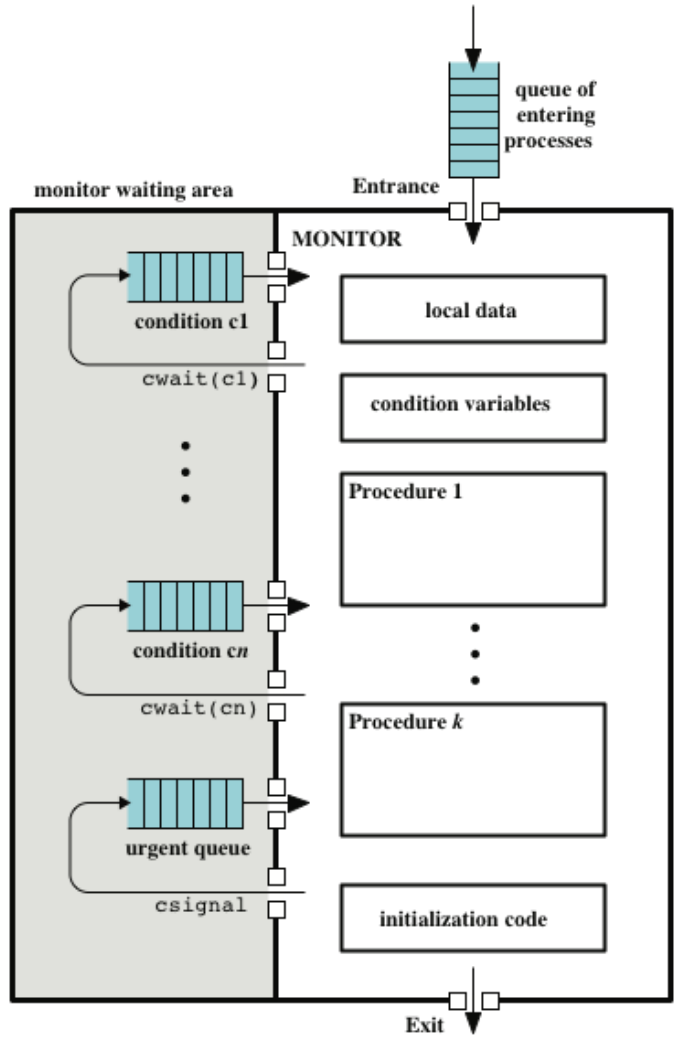


Figure 5.18 Structure of a Monitor

Condition variables

- Allow threads to **wait for a condition on shared data**, such as a queue becoming nonempty.
- Used within a monitor or with a **mutex** that protects the shared data.
- **cwait()** atomically releases the lock and blocks the calling thread. The thread must regain the lock before returning from wait().
- **csignal()** notifies a waiting thread; **broadcast()** notifies all waiting threads.
- Notifications are **not saved**: signaling when no threads are waiting has no effect.
- A condition variable does not store or test the condition—the program checks the shared data.

atomic operation	A function or action implemented as a sequence of one or more instructions that appears to be indivisible; that is, no other process can see an intermediate state or interrupt the operation. The sequence of instruction is guaranteed to execute as a group, or not execute at all, having no visible effect on system state. Atomicity guarantees isolation from concurrent processes.
critical section	A section of code within a process that requires access to shared resources and that must not be executed while another process is in a corresponding section of code.
deadlock	A situation in which two or more processes are unable to proceed because each is waiting for one of the others to do something.
livelock	A situation in which two or more processes continuously change their states in response to changes in the other process(es) without doing any useful work.
mutual exclusion	The requirement that when one process is in a critical section that accesses shared resources, no other process may be in a critical section that accesses any of those shared resources.
race condition	A situation in which multiple threads or processes read and write a shared data item and the final result depends on the relative timing of their execution.
starvation	A situation in which a runnable process is overlooked indefinitely by the scheduler; although it is able to proceed, it is never chosen.